Unit 6
File I/O

FILE

- File is a logical representation of data i.e. file is a collection of record.
- It does not describe how information is stored or retrieved in File or not change content of File.
- To manipulate file's attribute (properties of file)
- It is a <u>Class</u> which present in <u>java.io.*;</u> Package

File class

Constructors

- **File(String directoryPath)**
- **File(String directoryPath, String filename)**
- **File(File dirObj, String filename)**
- **File(URI uriObj)**

Method:

public boolean createNewFile()

Example

```
1. File f1=new File( “Demo.txt” );

boolean b=f1.createNewFile();

2. File f2=new File( “C:\java\bin\myFile.txt” );

boolean b=f2.createNewFile();
```

File class

Method

public boolean mkdir()

Example

```
3. File f=new File( “c:\syif” );
boolean b=f.mkdir();
4. File f3=new File (f, “Hello.txt”);
boolean value=f3.createNewFile();
```

Sample code for creation of new “File”

```
import java.io.*;
public class CreateFile
{
  public static void main(String args[])
  {
      try {
     File f = new File("F:\program.txt");

     if (f.createNewFile())
                System.out.println("File created");
          else
                System.out.println("File already exists");
     }
     catch (Exception e) {
        System.err.println(e);
     }
 }
}
```

Sample code for creation of new “Folder”

```
import java.io.*;
public class demo
{
  public static void main(String args[])
  {
        File f = new File("F:\program");
        if (f.mkdir())
        {
              System.out.println("Directory is created");
        }
        else {
              System.out.println("Directory cannot be created");
        }
      }
}
```

Methods of FILE class	
Method	**Description**
boolean mkdir()	Create a Directory at run time
boolean createNewFile()	Create a new file at run time
String getName()	Name of File
String getPath()	Whole path of File
String getParent()	Name of parent directory
boolean exists()	Is File present or not
boolean canRead()	Can user read this file
boolean canWrite()	Can user write in File
boolean isDirectory()	Is given File is directory / not file
boolean isFile()	Is given File is File / not directory
boolean delete()	Delete File and return true on deletion
boolean renameTo(name)	Rename to new name & return true
boolean isHidden()	Is File is hidden or not
int length()	Returns size of File in bytes

sample code on file

```
//  File class operations
import java.io.File;
class FileClass
{
    public static void main(String args[])
    {
        File f = new File("c://boot.ini");
        System.out.println("isFile:"+f.isFile());
        System.out.println("Name:"+f.getName());
        System.out.println("Size:"+f.length());
        System.out.println("Path:"+f.getPath());
        System.out.println("Parent:"+f.getParent());
        System.out.println("Hidden:"+f.isHidden());
    }
}
```

Program 7.1 File class operations

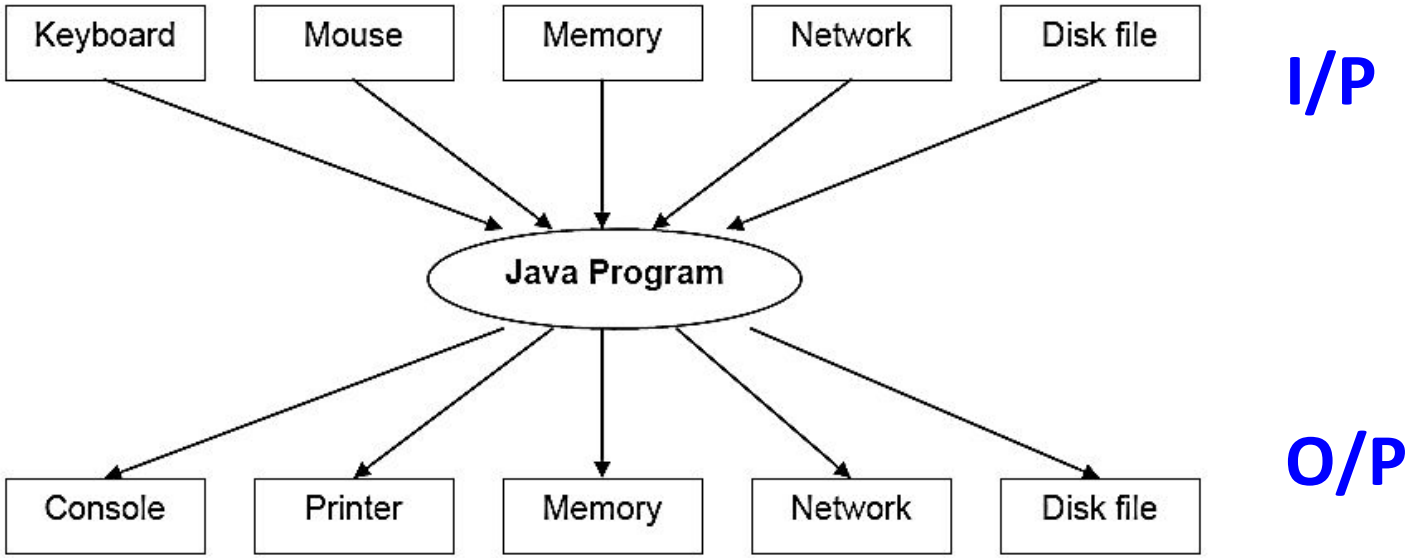
Output:

```
isFile:true
Name:boot.ini
Size:194
Path:c:\boot.ini
Parent:c:\
Hidden:true
```

Stream class

Stream : It is flow of data

Regarding file processing, input refers to the flow of data inside the program and output means flow of data outside of the program.



I/P

O/P

Fig. 7.1 data flow of a Java program

Classification of Stream class

- Java uses the concept of stream to represent the ordered sequence of data.

- Stream classes available in java.io package

- Java streams are classified into two categories namely, input stream and output stream.

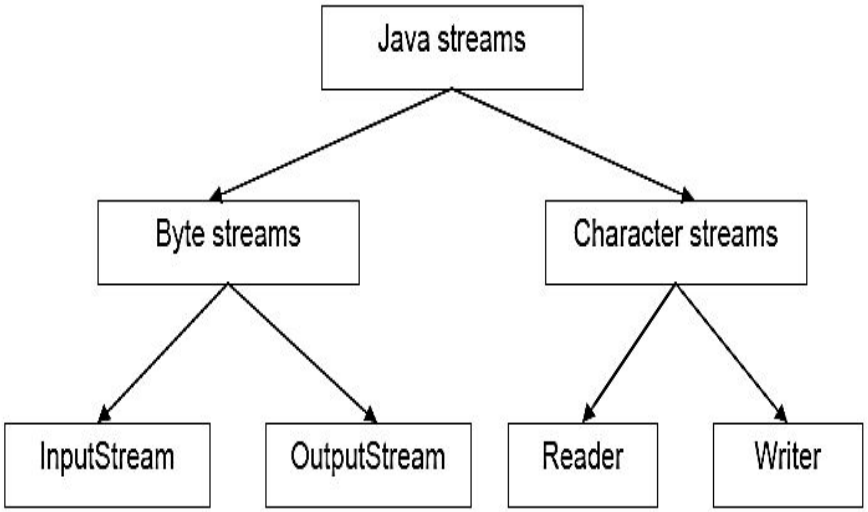


Fig. 7.2 classification Java stream classes

Classification of stream class

- **Bytestream** :Perform input output operations on <u>byte</u> or <u>binary</u> object.
  - **Inputstream** : use to read 8 bit bytes.
  - **Outputstream** : use to write 8 bit bytes.
- **Characterstream:** Perform input output operations on <u>character</u> or <u>string</u>.
  - **Reader** : perform input operations i.e read characters
  - **Writer** : use to perform output operations i.e write characters.

Classification of Input stream class

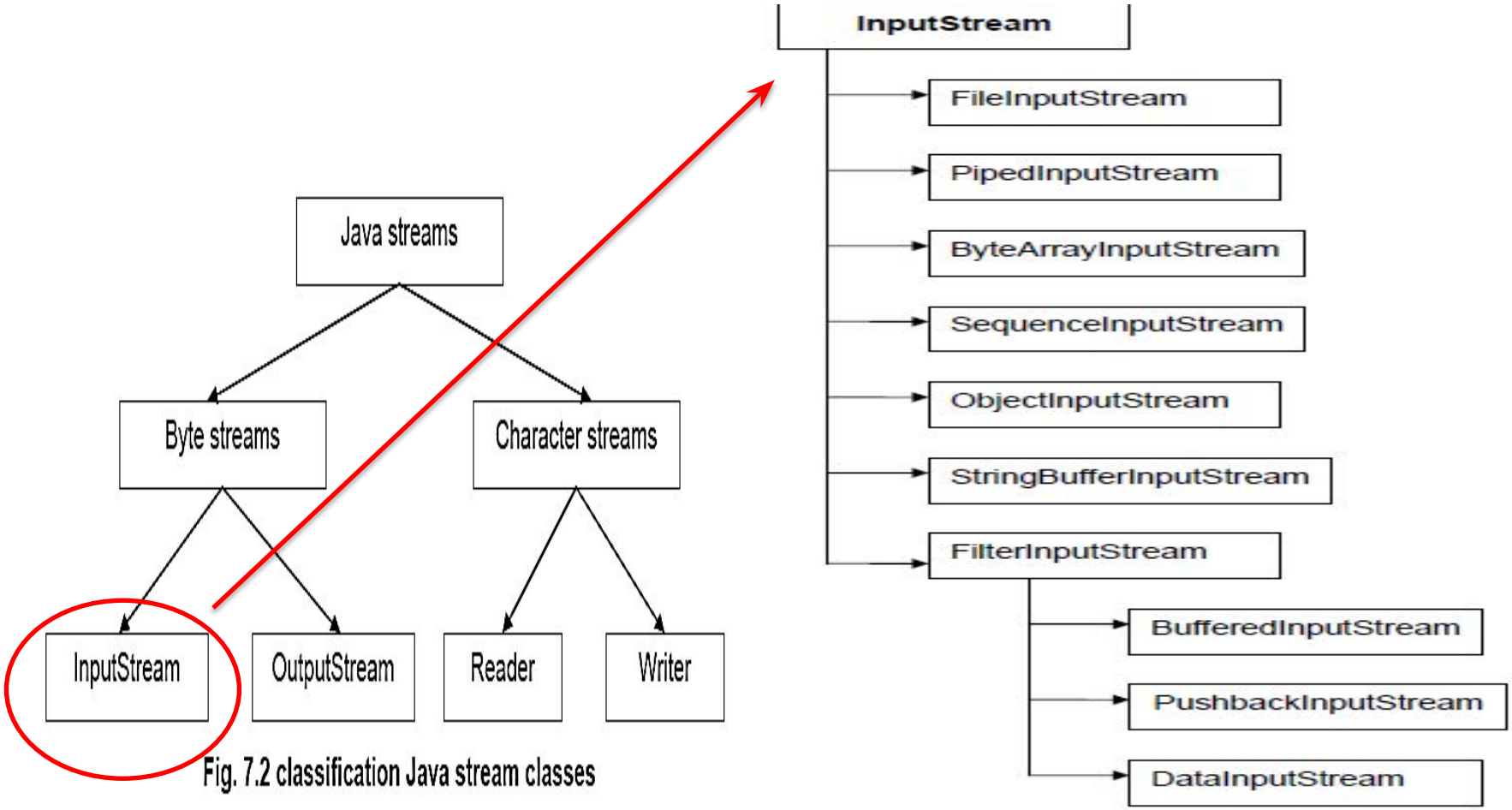


Fig. 7.2 classification Java stream classes

Methods of InputStream class

Sr. No	Method name	Use
1	int available()	It return currently available bytes of data in file
2	void close()	It closes the input stream.
3	int read()	It returns an integer representation of the next available byte of input. It returns value –1 when the end of the file is encountered.
4	int read(byte buffer[])	It attempts to read up to 'buffer.length' bytes into byte array 'buffer' and returns the actual number of bytes that were successfully read. It returns –1 when the end of the file is encountered.

FileInputStream class

- **The FileInputStream class creates an InputStream by which we can read bytes from a file.**

- It's having two most common **constructors:**

Syntax…

FileInputStream(String filepath)

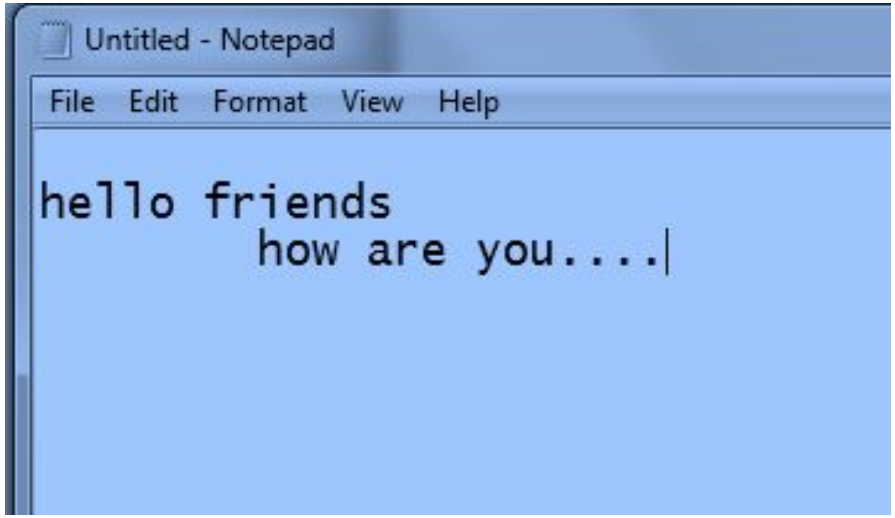
FileInputStream(File fileobj)

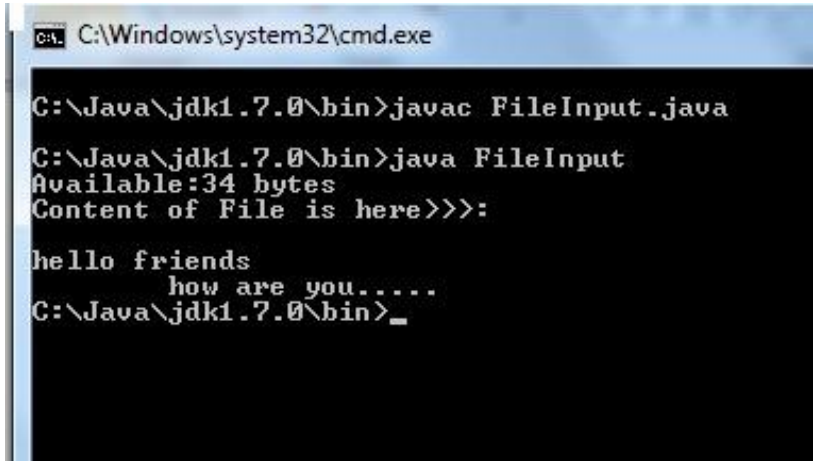
Eg: FileInputStream f1=new FileInputStream("demo.txt");

Sample code to read data into file using FileInputStream class

```
// before executing program you create "new.txt" file in \bin folder
// and write any thing in it.....

import java.io.*;
class FileInput
{
    public static void main(String args[]) throws Exception
    {
        FileInputStream f = new FileInputStream("new.txt");

        int size = f.available();

        System.out.println("Available:"+ size +" bytes");
        System.out.println("Content of File is here>>>:");

              for(int i=0;i<size;i++)
              {
                      System.out.print((char) f.read());
              }

      f.close();
    }
}
```

OUTPUT


Untitled - Notepad
File Edit Format View Help
hello friends
how are you....


C:\Windows\system32\cmd.exe
C:\Java\jdk1.7.0\bin>javac FileInput.java
C:\Java\jdk1.7.0\bin>java FileInput
Available:34 bytes
Content of File is here>>>:
hello friends
how are you.....
C:\Java\jdk1.7.0\bin>

Classification of Output Stream class

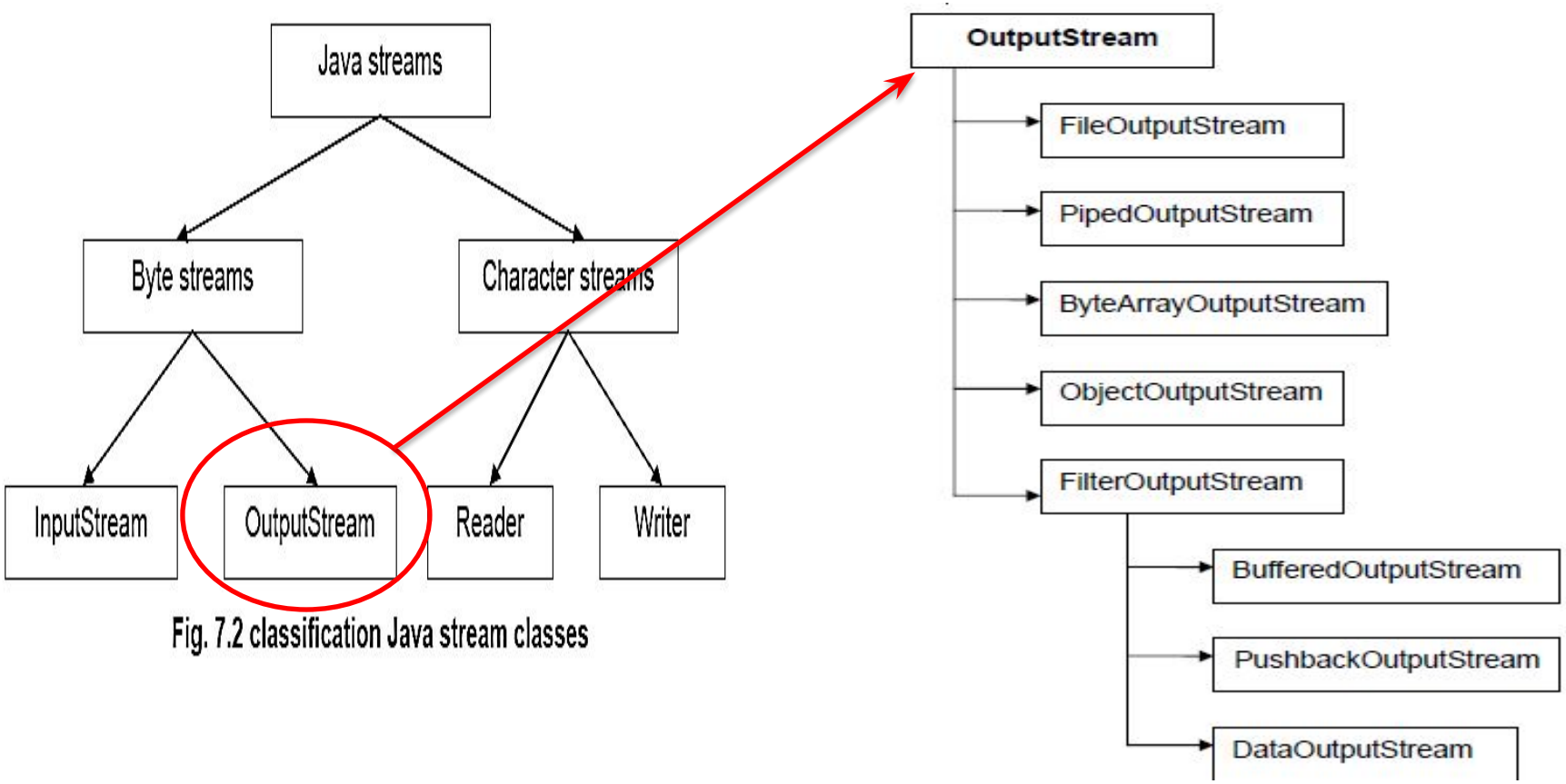


Fig. 7.2 classification Java stream classes

Methods of OutputStream class

Sr. No	Method name	Use
1	void close()	It closes the output stream
2	void flush()	This method finalizes the output state so that any buffers are cleared.
3	void write(int b)	It writes a single byte to an output stream.
4	void write(byte buffer[])	it writes a complete array of bytes ‘buffer’ to an output stream

FileOutputStream class

- **The FileOutputStream class creates an OutputStream by which we can write bytes in a file.**

- It's having 4 most common **constructors:**

Syntax…

FileOutputStream(String filepath)
FileOutputStream(File fileobj)
FileOutputStream(String filePath, boolean append)
FileOutputStream(File fileObj, boolean append)

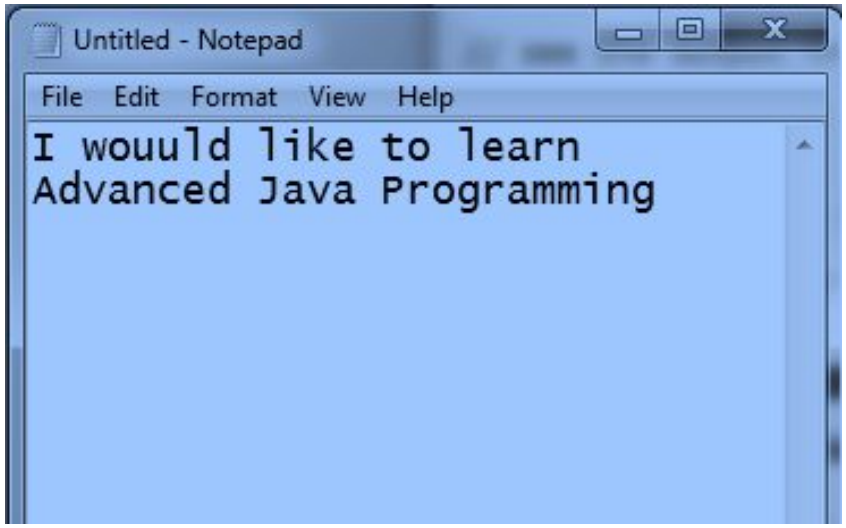
Throw FileNotFoundException exception

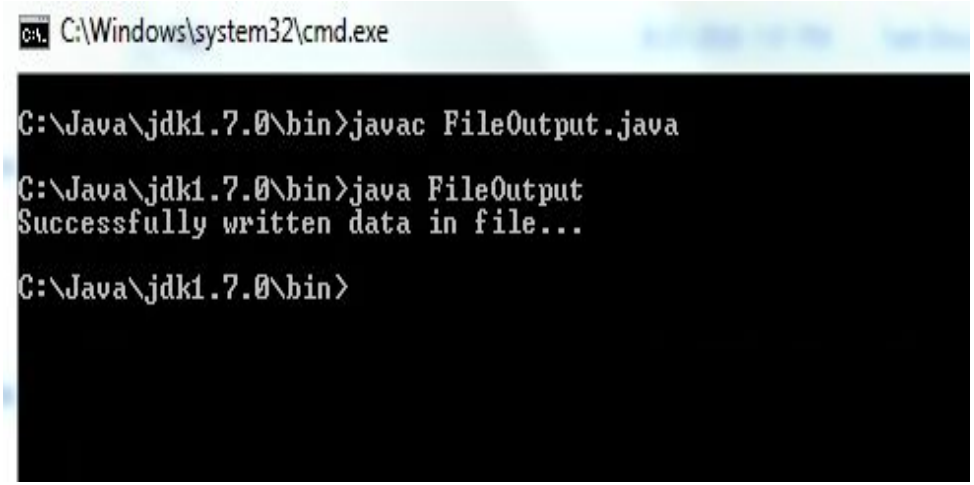
Eg: **FileOutputStream f1=new FileOutputStream("demo.txt");**

Sample code to write data into file using FileOutputStream class

```
// see the output in "file1.txt" in \bin after executing program

  import java.io.*;
  class FileOutput
  {
      public static void main(String args[]) throws Exception
      {
          String str = "I would like to learn Advanced"
                       + " Java Programming ;

          byte buf[] = str.getBytes(); //statement1

          OutputStream f1 = new FileOutputStream("file1.txt");
                                 // automatically create and write file

                for (int i=0; i < buf.length; i++)
                {
                        f1.write(buf[i]);
                }
                System.out.println("Successfully written data in file...");
          f1.close();

      }
  }
```

OUTPUT


Untitled - Notepad
File Edit Format View Help
I wouuld like to learn
Advanced Java Programming


C:\Windows\system32\cmd.exe
C:\Java\jdk1.7.0\bin>javac FileOutput.java
C:\Java\jdk1.7.0\bin>java FileOutput
Successfully written data in file...
C:\Java\jdk1.7.0\bin>

Classification of Reader class

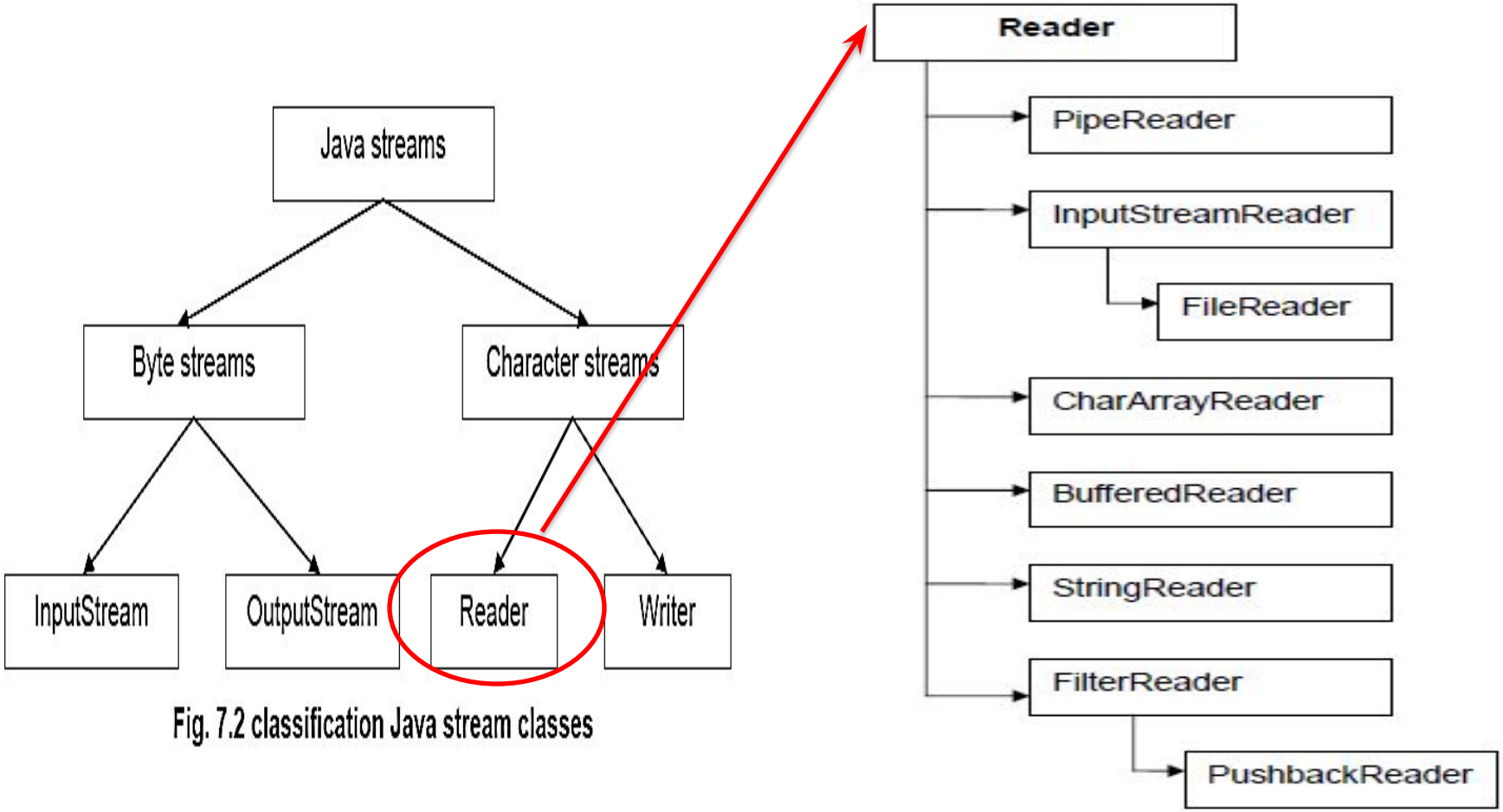


Fig. 7.2 classification Java stream classes

FileReader class

- **The FileReader class used to read data char. by char. from file.**

- It's having two most common **constructors:**

Syntax…

FileReader(String filepath)

FileReader(File fileobj)

Eg: **FileReader FR=new FileReader("demo.txt");**

Sample code to read data from file using FileReader class

```
// read data from file using FileReader class
import java.io.*;
class FileRead
{
  public static void main(String args[])throws Exception
  {
        FileReader fr = new FileReader("hello.txt");
        int ch;

        while((ch =fr.read())!= -1)
        {
                System.out.print((char) ch);
        }

        fr.close();
  }
}
```

Classification of Writer class

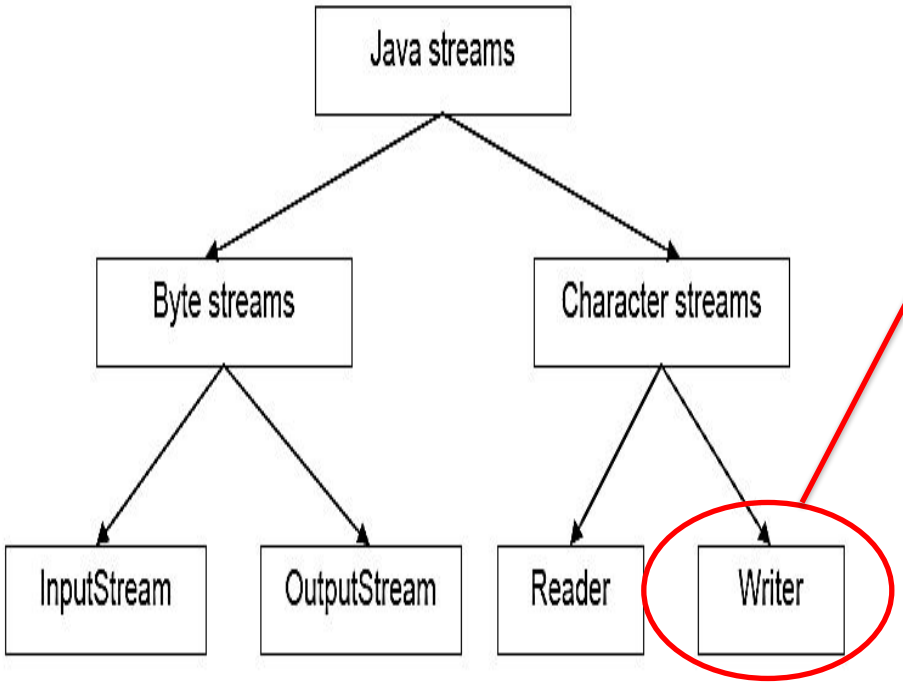
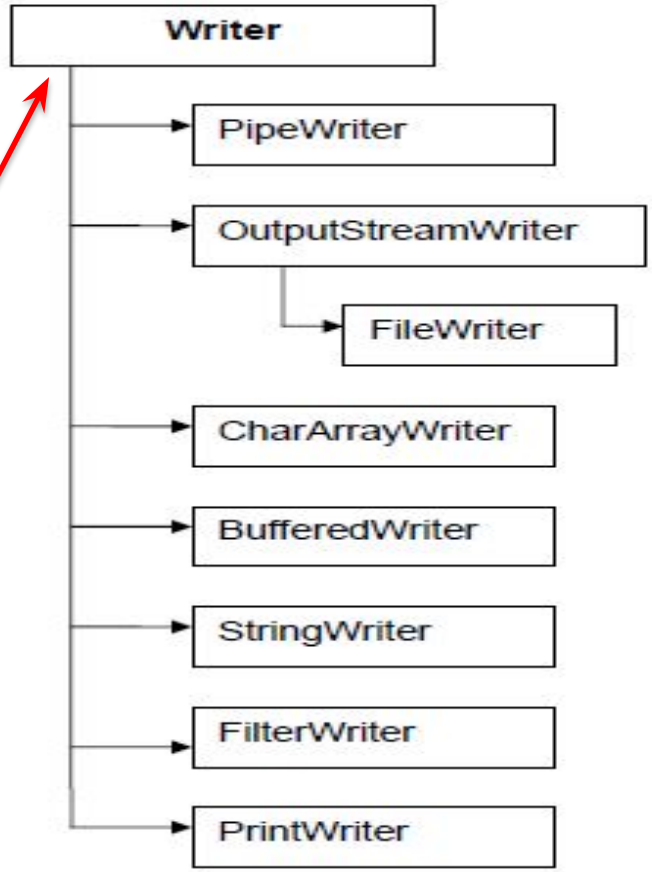


Fig. 7.2 classification Java stream classes



FileWriter class

- **The FileWriter class used to write data char. by char. into file.**

- It's having two most common **constructors:**

Syntax…

FileWriter (String filepath)

FileWriter (File fileobj)

Eg:

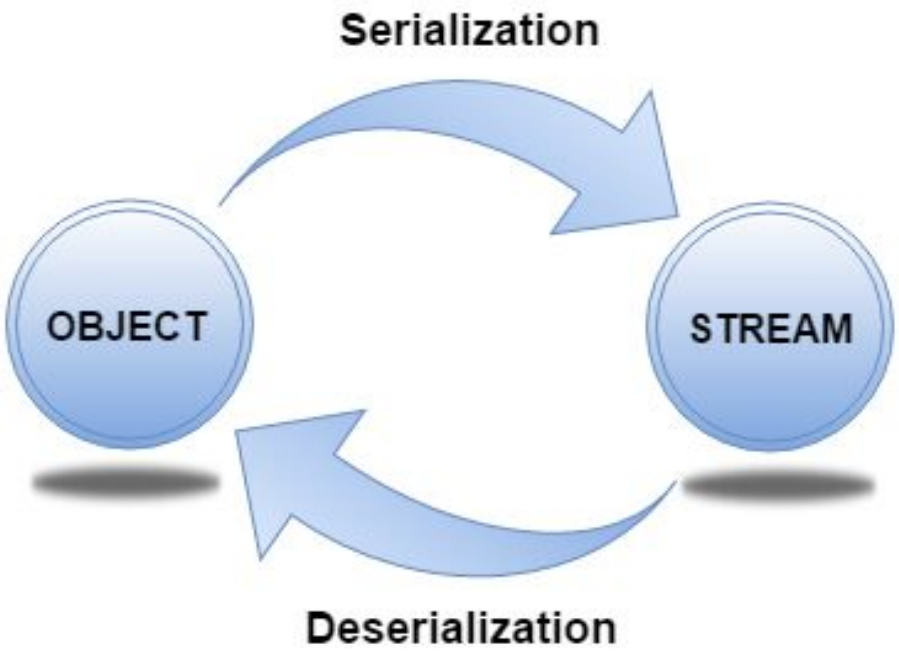
FileWriter FR=new FileWriter ("demo.txt");

Sample code to write data into file using FileWriter class

```
// Demonstrating FileWriter.
import java.io.FileWriter;
class FileWrite
{
public static void main(String args[])throws Exception
{
        String str = "I am addicted to Programming";

        char buf[] = str.toCharArray();
        int len = buf.length;

        FileWriter f1 = new FileWriter("hello.txt");

        for(int i = 0;i<len;i++)
        {
                f1.write(buf[i]);
        }
        System.out.println("Sucessfully write data...");
        f1.close();
}
}
```

Object Serialization

- Process of converting object representation into stream of bytes is known as **Serialization.**
- Process of converting stream of bytes into object representation is known as **De-serialization.**



Object Serialization

- In Serialization: where an object can be represented as a *sequence of bytes* that includes the *object's data as well as information about the object's type* and the types of data stored in the object.
- After a serialized object has been written into a file, it can be read from the file.

- Once the object has been written, then byte stream can also convert into object representation….
- A class must implements "Serializable" interface whose object user want to serialize.

Serialization

Classes required for serialization/deserialization:

- **FileInputStream / FileOutputStream**
- **ObjectInputStream**
  - It deserializes the objects
- **ObjectOutputStream**
  - It is used to write object states to the file.

Methods required for serialization:

❖ public final void writeObject(Object o) throws IO Exception

Method required for deserialization:

❖ public final Object readObject() throws IOException, ClassNotFoundException

Sample code on object serialization

```
class Chapter implements Serializable
{
        String s;
        int i;
        public Chapter(String str, int in)
        {
                s = str;
                i = in;
        }
        public void show()
        {
                System.out.println("String: "+ s);
                System.out.println("Value : "+ i);
        }
}
```

Continue…..

```
public class Serialization
{
    public static void main(String args[])
        throws Exception
    {
        // Object serialization
        Chapter obj1 = new Chapter("Amit", 96); //statement1
        System.out.println("Object to be serialized:");
        obj1.show();
        FileOutputStream file1 =
                        new FileOutputStream("object.txt");
        ObjectOutputStream out1 =
                        new ObjectOutputStream(file1);
        out1.writeObject(obj1);    //statement2
        out1.flush();              //statement3
        out1.close();              //statement4

        // Object deserialization
        Chapter obj2;              //statement5
        FileInputStream file2 =
                        new FileInputStream("object.txt");

        ObjectInputStream out2 =
                        new ObjectInputStream(file2);
        obj2 = (Chapter)out2.readObject();    //statement6
        out2.close();                         //statement7
        System.out.println("\nObject deserialized:");
        obj2.show();
    }
}
```

End of Chap 6

Thank You…